

CUDA C++ By Example

Session 6: Optimizations III + Libraries

Overview

Last time:

Deep-dive on memory-bound kernels

This time:

Some more thoughts on compute-bound optimizations

Topics from Discussions Last Week

- How expensive is it to do sin/exp/cos on a GPU?
 - Can you explain the "roofline" model?
- How to deal with small kernels on GPU?
- When should I consider reduced precision arithmetic?
- How to deal with load imbalance / divergence in a kernel?
- How to generate sparse matrices on GPU?

Instruction Latencies

- How expensive is it to do _____ on a GPU?
 - basic arithmetic operations
 - transcendental functions
 - bitwise operations
 - ...
- Not all basic operations are equivalent!

Instruction Latencies

- See: 06_profiling/instruction_latencies directory
- Try building and running the executable `instruction_latencies`
- Which instructions are the most expensive?
- How does the precision / signedness affect the runtime?
- What is the purpose of the conditional assignment at the end of the kernel?

Instruction Latencies

		i32:	x + i32: 0.014336 ms
no-op: 0.004096 ms			x * i32: 0.338944 ms
			i32 % x: 10.3844 ms
f32:			x % i32: 1.84115 ms
x + f32: 0.339968 ms			i32 / x: 11.2015 ms
x * f32: 0.340992 ms			x / i32: 1.42746 ms
x / f32: 20.4196 ms		i64:	
f32 / x: 3.4775 ms			x + i64: 0.014336 ms
f32 *	x + f32: 0.338944 ms		x * i64: 1.36602 ms
	sin(x): 8.11933 ms		i64 % x: 12.3668 ms
	tan(x): 9.94202 ms		x % i64: 6.08563 ms
	exp(x): 3.44678 ms		i64 / x: 12.5204 ms
	log(x): 7.43629 ms		x / i64: 4.93056 ms
	asin(x): 8.43059 ms	u32:	
f64:			x + u32: 0.017408 ms
	x + f64: 0.66864 ms		x * u32: 0.338944 ms
	x * f64: 0.668672 ms		u32 % x: 12.6362 ms
	x / f64: 33.5329 ms		x % u32: 1.51142 ms
	f64 / x: 8.82381 ms		u32 / x: 10.6383 ms
f64 *	x + f64: 0.668672 ms		x / u32: 1.0967 ms
	sin(x): 17.8944 ms	u64:	
	tan(x): 19.4673 ms		x + u64: 0.021504 ms
	exp(x): 13.2977 ms		x * u64: 1.36602 ms
	log(x): 6.79526 ms		u64 % x: 12.4211 ms
	asin(x): 16.0727 ms		x % u64: 4.67046 ms
			u64 / x: 12.3382 ms
			x / u64: 3.2809 ms

Titan V

no-op: 0.003232 ms		i32:	x + i32: 0.006144 ms
			x * i32: 0.139264 ms
			i32 % x: 4.49533 ms
			x % i32: 0.792544 ms
f32:			i32 / x: 4.91114 ms
	x + f32: 0.141312 ms		x / i32: 0.622592 ms
	x * f32: 0.141312 ms	i64:	
	x / f32: 9.13002 ms		x + i64: 0.006144 ms
	f32 / x: 1.56157 ms		x * i64: 0.550912 ms
f32 *	x + f32: 0.141312 ms		i64 % x: 5.58915 ms
	sin(x): 3.16829 ms		x % i64: 2.68186 ms
	tan(x): 3.61059 ms		i64 / x: 5.62278 ms
	exp(x): 1.51347 ms		x / i64: 2.10432 ms
	log(x): 3.12362 ms	u32:	
	asin(x): 3.30339 ms		x + u32: 0.006176 ms
f64:			x * u32: 0.14128 ms
	x + f64: 2.1801 ms		u32 % x: 4.56294 ms
	x * f64: 2.18214 ms		x % u32: 0.65536 ms
	x / f64: 51.0075 ms		u32 / x: 4.56499 ms
	f64 / x: 18.7628 ms		x / u32: 0.4848 ms
f64 *	x + f64: 2.18832 ms	u64:	
	sin(x): 43.5046 ms		x + u64: 0.006144 ms
	tan(x): 57.7393 ms		x * u64: 0.550912 ms
	exp(x): 37.0708 ms		u64 % x: 5.59206 ms
	log(x): 5.64531 ms		x % u64: 1.93536 ms
	asin(x): 66.4105 ms		u64 / x: 5.6248 ms
			x / u64: 1.30355 ms

RTX 5090

Instruction Latencies

no-op: 0.004096 ms	
f32:	
x + f32: 0.339968 ms	
x * f32: 0.340992 ms	
x / f32: 20.4196 ms	
f32 / x: 3.4775 ms	
f32 * x + f32: 0.338944 ms	
sin(x): 8.11933 ms	
tan(x): 9.94202 ms	
exp(x): 3.44678 ms	
log(x): 4.43629 ms	
asin(x): 4.43629 ms	
f64:	
x + f64: 0.66864 ms	
x * f64: 0.668672 ms	
x / f64: 33.5329 ms	
f64 / x: 8.82381 ms	
f64 * x + f64: 0.668672 ms	
sin(x): 17.8944 ms	
tan(x): 19.4673 ms	
exp(x): 13.2977 ms	
log(x): 6.79526 ms	
asin(x): 16.0727 ms	
i32:	
x + i32: 0.014336 ms	
x * i32: 0.338944 ms	
i32 % x: 10.3844 ms	
x % i32: 1.84115 ms	
i32 / x: 11.2015 ms	
x / i32: 1.42746 ms	
i64:	
x + i64: 0.014336 ms	
x * i64: 1.36602 ms	
i64 % x: 12.3668 ms	
x % i64: 6.08563 ms	
i64 / x: 12.5204 ms	
x / i64: 4.93056 ms	
u32:	
x + u32: 0.017408 ms	
x * u32: 0.338944 ms	
u32 % x: 12.6362 ms	
x % u32: 1.51142 ms	
u32 / x: 10.6383 ms	
x / u32: 1.0967 ms	
u64:	
x + u64: 0.021504 ms	
x * u64: 1.36602 ms	
u64 % x: 12.4211 ms	
x % u64: 4.67046 ms	
u64 / x: 12.3382 ms	
x / u64: 3.2809 ms	

no-op: 0.003232 ms	
f32:	
x + f32: 0.141312 ms	
x * f32: 0.141312 ms	
x / f32: 9.13002 ms	
f32 / x: 1.56157 ms	
f32 * x + f32: 0.141312 ms	
sin(x): 3.16829 ms	
tan(x): 3.61059 ms	
exp(x): 1.51347 ms	
log(x): 3.12262 ms	
asin(x): 3.12262 ms	
f64:	
x + f64: 2.1801 ms	
x * f64: 2.18214 ms	
x / f64: 51.0075 ms	
f64 / x: 18.7628 ms	
f64 * x + f64: 2.18832 ms	
sin(x): 43.5046 ms	
tan(x): 57.7393 ms	
exp(x): 37.0708 ms	
log(x): 5.64531 ms	
asin(x): 66.4105 ms	
i32:	
x + i32: 0.006144 ms	
x * i32: 0.139264 ms	
i32 % x: 4.49533 ms	
x % i32: 0.792544 ms	
i32 / x: 4.91114 ms	
x / i32: 0.622592 ms	
i64:	
x + i64: 0.006144 ms	
x * i64: 0.550912 ms	
i64 % x: 5.58915 ms	
x % i64: 2.68186 ms	
i64 / x: 5.62278 ms	
x / i64: 2.10432 ms	
u32:	
x + u32: 0.006176 ms	
x * u32: 0.14128 ms	
u32 % x: 4.56294 ms	
x % u32: 0.65536 ms	
u32 / x: 4.56499 ms	
x / u32: 0.4848 ms	
u64:	
x + u64: 0.006144 ms	
x * u64: 0.550912 ms	
u64 % x: 5.59206 ms	
x % u64: 1.93536 ms	
u64 / x: 5.6248 ms	
x / u64: 1.30355 ms	

What's going on with floating point division?

Instruction Latencies

```
for (int k = 0; k < 100000; k++) { local = f(local); }
```

```
std::cout << std::setw(20) << " x / f64: " << runtime_ms(  
    [] __device__ (double x){ return x / 42.0; },  
    d_f64  
) << " ms" << std::endl;
```

f64:

x + f64: 0.66864 ms
x * f64: 0.668672 ms
x / f64: 33.5329 ms
f64 / x: 8.82381 ms

```
std::cout << std::setw(20) << " x / f64: " << runtime_ms(  
    [] __device__ (double x){ return x / 1.0001; },  
    d_f64  
) << " ms" << std::endl;
```

f64:

x + f64: 0.667648 ms
x * f64: 0.667648 ms
x / f64: 8.82483 ms
f64 / x: 8.82381 ms

Instruction Latencies

Integer Division: Demo

See also: <https://libdivide.com/> for more theory

Instruction Latencies

```
template < typename callable, typename T >
__global__ void kernel_template(callable f, T * data) {
    volatile T local = *data;

    for (int k = 0; k < 100000; k++) {
        local = f(local);
    }

    //if (threadIdx.x > 512) {
    //    data[0] = local; // never executes
    //}
}
```

If we comment out the kernel's output ...

Instruction Latencies

```
template < typename callable, typename T >
__global__ void kernel_template(callable f, T * data) {
    volatile T local = *data;

    for (int k = 0; k < 100000; k++) {
        local = f(local);
    }

    //if (threadIdx.x > 512) {
    //    data[0] = local; // never executes
    //}
}
```

x + f32:	0.004096 ms
x * f32:	0.003072 ms
x / f32:	20.1308 ms
f32 / x:	3.46214 ms
f32 * x + f32:	0.004096 ms
sin(x):	8.12032 ms
tan(x):	10.1079 ms
exp(x):	0.004096 ms
log(x):	0.004096 ms
asin(x):	0.004096 ms

If we comment out the kernel's output ...

The compiler optimizes away
the entire kernel!
(some of the time?)

Instruction Latencies

NCU report: `instruction_latencies_trig.ncu-rep`

Roofline Model

The roofline model is (in my mind) another way of describing / plotting arithmetic intensity.

$$\text{time} := \max((\text{kernel ops})/(\text{GPU FLOP/s}), (\text{kernel bytes})/(\text{GPU bytes/s}))$$

Let's check it out with an example:
[06_profiling/instruction_latencies/modified_axpy.cu](#)

Roofline Model

The roofline model is (in my mind) another way of describing / plotting arithmetic intensity.

Let's check it out with an example:
`06_profiling/instruction_latencies/modified_axpy.cu`

Before we run the application: what do we expect the runtimes to look like?

Roofline Model

```
$ ./modified_axpy
```

```
regular axpy: 0.167936 ms, 599.415 GB/s
```

```
modified (kmax = 0) axpy: 0.166912 ms, 603.092 GB/s
```

```
modified (kmax = 1) axpy: 0.167936 ms, 599.415 GB/s
```

```
modified (kmax = 2) axpy: 0.167936 ms, 599.415 GB/s
```

```
modified (kmax = 3) axpy: 0.167936 ms, 599.415 GB/s
```

```
modified (kmax = 4) axpy: 0.169984 ms, 592.193 GB/s
```

```
modified (kmax = 5) axpy: 0.169984 ms, 592.193 GB/s
```

```
modified (kmax = 6) axpy: 0.169984 ms, 592.193 GB/s
```

```
modified (kmax = 7) axpy: 0.18944 ms, 531.373 GB/s
```

```
modified (kmax = 8) axpy: 0.20992 ms, 479.532 GB/s
```

```
modified (kmax = 9) axpy: 0.23552 ms, 427.409 GB/s
```

```
modified (kmax = 10) axpy: 0.26112 ms, 385.506 GB/s
```

```
modified (kmax = 11) axpy: 0.285696 ms, 352.344 GB/s
```

```
modified (kmax = 12) axpy: 0.3072 ms, 327.68 GB/s
```

```
modified (kmax = 13) axpy: 0.331776 ms, 303.407 GB/s
```

```
modified (kmax = 14) axpy: 0.357376 ms, 281.673 GB/s
```

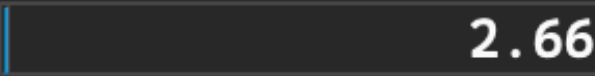
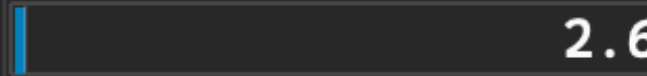
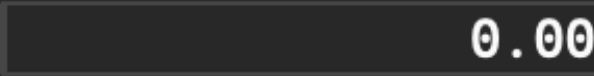
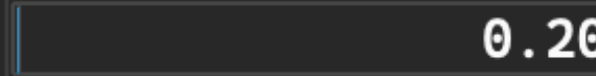
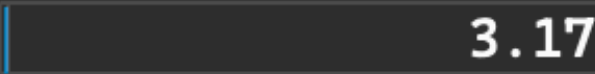
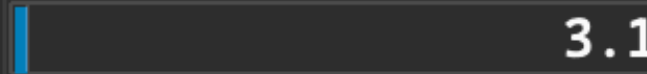
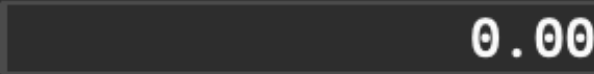
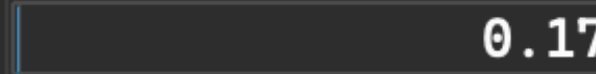
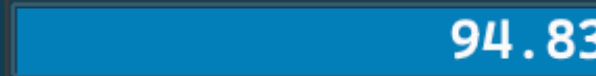
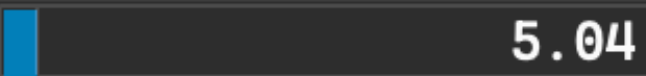
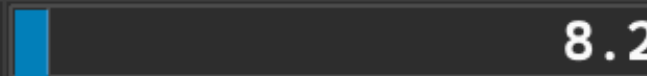
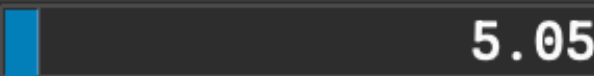
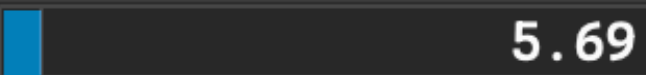
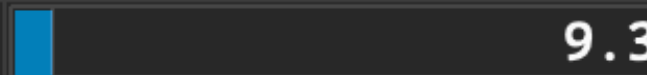
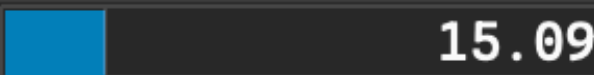
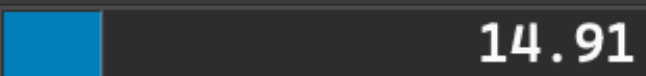
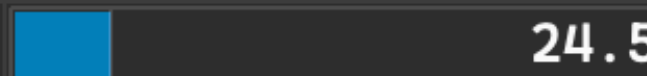
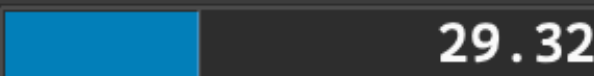
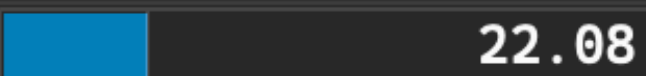
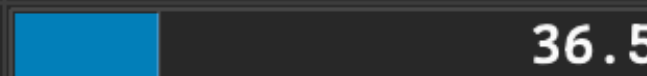
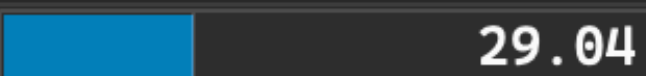
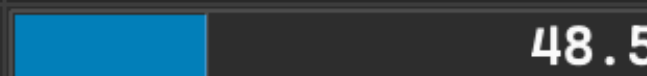
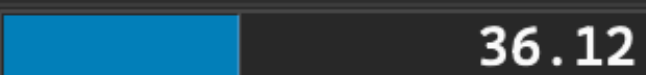
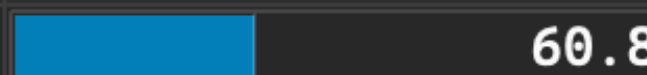
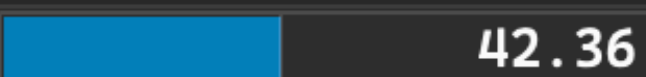
```
modified (kmax = 15) axpy: 0.381952 ms, 263.55 GB/s
```

Roofline Model

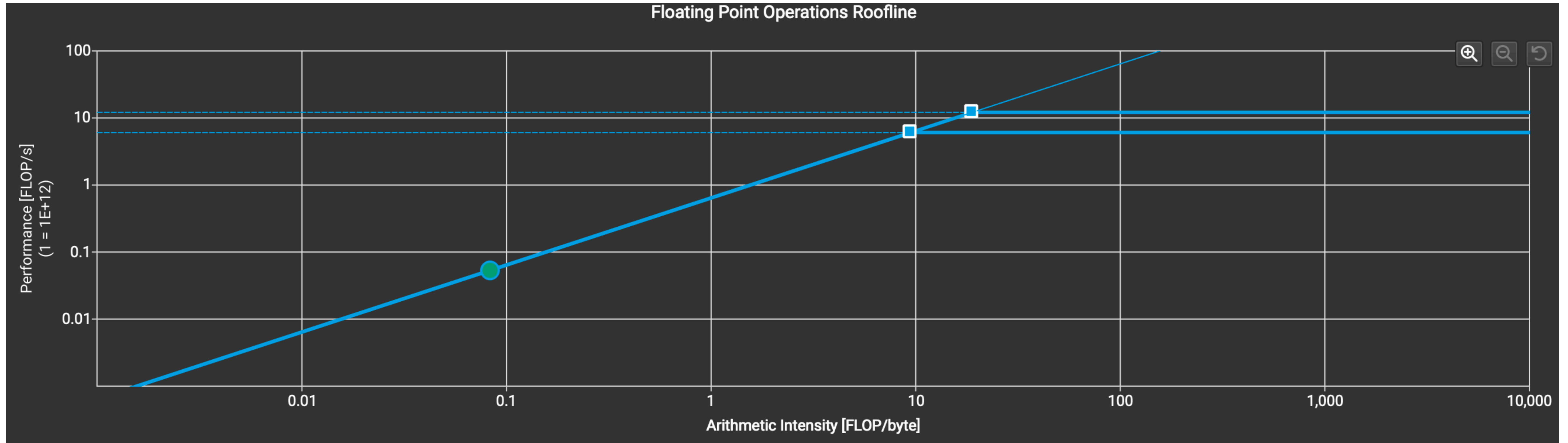
Let's profile with NCU:

```
ncu --set full -o modified_axpy -f --clock-control none ./modified_axpy
```


Roofline Model

0		98.75	regular_axy	regular_axy(double, co...		2.66		2.62		0.00		0.20
1		98.75	modified_axy	modified_axy(double, ...		3.17		3.13		0.00		0.17
2		5.17	regular_axy	regular_axy(double, co...		163.97		8.48		4.22		94.83
3		5.04	modified_axy	modified_axy(double, ...		163.52		8.25		5.05		94.96
4		5.69	modified_axy	modified_axy(double, ...		164.90		9.39		15.09		94.31
5		14.91	modified_axy	modified_axy(double, ...		164.80		24.57		29.32		93.94
6		22.08	modified_axy	modified_axy(double, ...		165.28		36.50		43.51		93.77
7		29.04	modified_axy	modified_axy(double, ...		167.26		48.57		57.17		92.58
8		36.12	modified_axy	modified_axy(double, ...		168.42		60.83		71.02		92.74
9		42.36	modified_axy	modified_axy(double, ...		169.95		71.99		83.37		90.62
10		44.13	modified_axy	modified_axy(double, ...		186.75		82.42		87.31		81.91
11		45.05	modified_axy	modified_axy(double, ...		209.02		94.17		89.30		73.32
12		45.05	modified_axy	modified_axy(double, ...		234.02		105.41		89.50		65.24
13		45.14	modified_axy	modified_axy(double, ...		259.04		116.94		89.60		59.00
14		45.25	modified_axy	modified_axy(double, ...		284.13		128.58		89.81		53.80
15		45.81	modified_axy	modified_axy(double, ...		306.18		140.27		90.90		49.98
16		45.84	modified_axy	modified_axy(double, ...		330.91		151.68		91.11		46.19
17		45.84	modified_axy	modified_axy(double, ...		356.22		163.29		91.15		42.93
18		45.88	modified_axy	modified_axy(double, ...		380.80		174.71		91.27		40.01

Roofline Model



Roofline Model

$\text{time} := \max((\text{kernel ops})/(\text{GPU FLOP/s}), (\text{kernel bytes})/(\text{GPU bytes/s}))$

Summary:

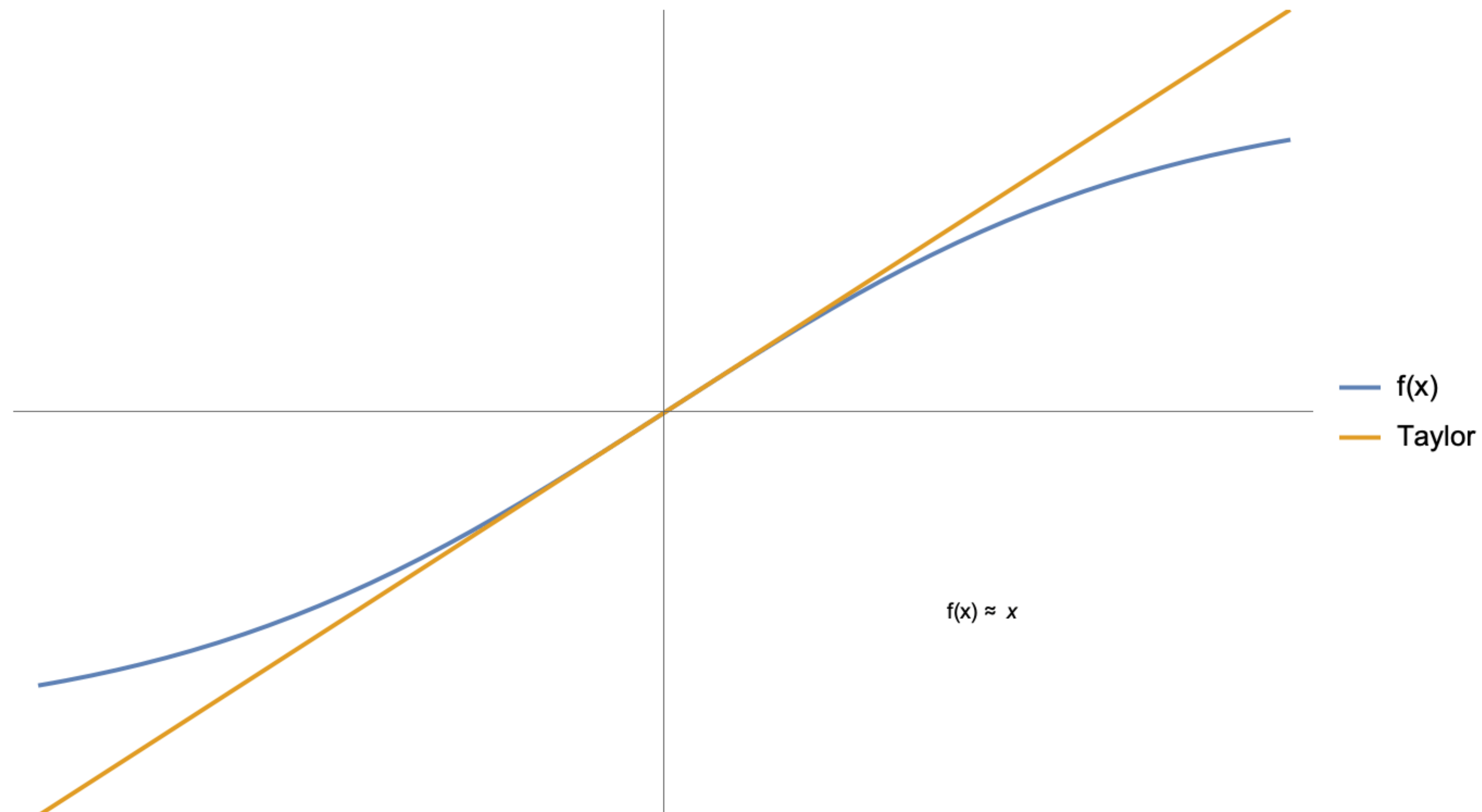
- if the kernel is strongly memory-bound, we can do more computation for "free"
- if the kernel is strongly compute-bound we can move more memory for "free"

Function Approximation

Given an expensive function $f(x)$,
can we find some cheaper way to approximate it?

Function Approximation

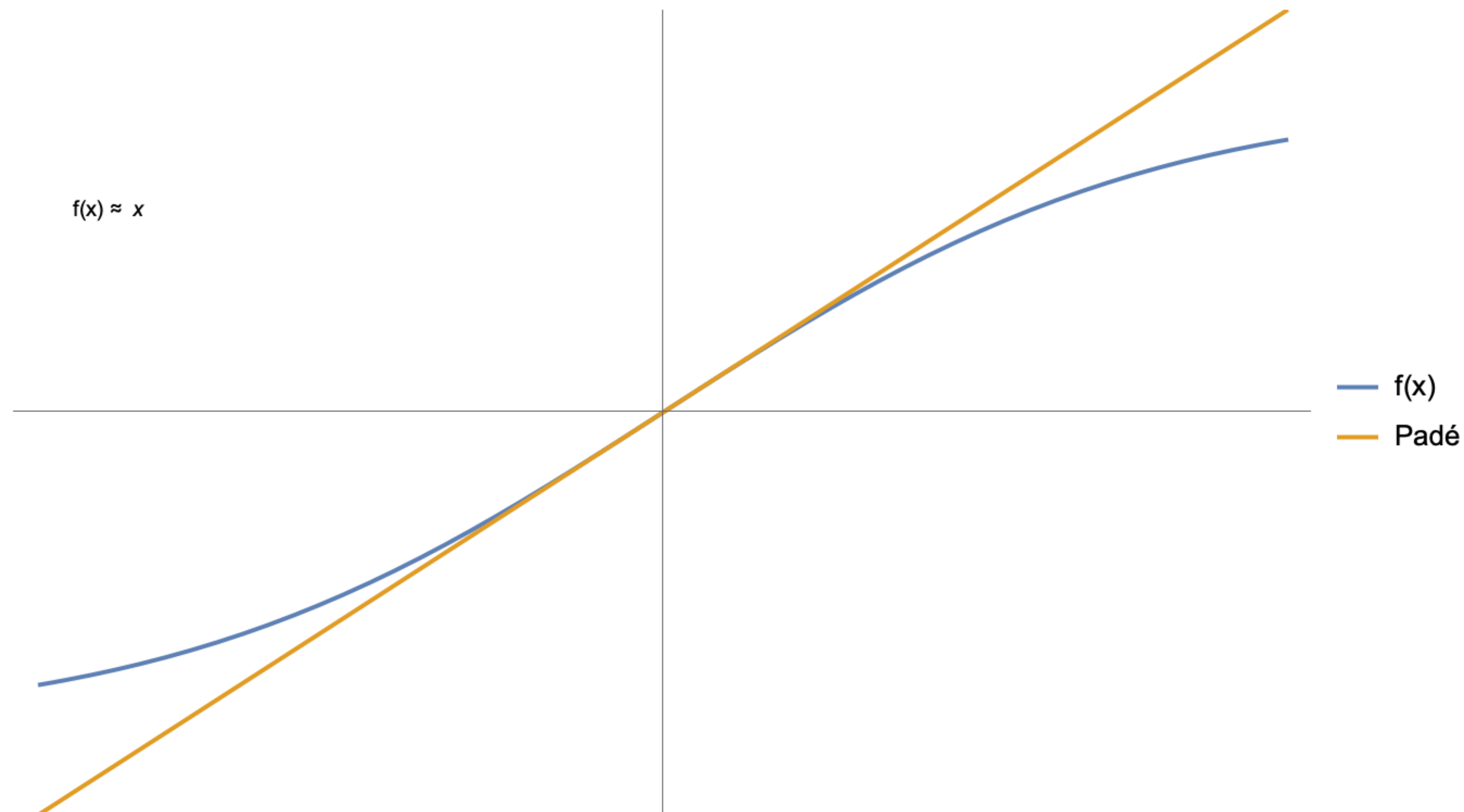
Given an expensive function $f(x)$,
can we find some cheaper way to approximate it?



Polynomial Approximations

Function Approximation

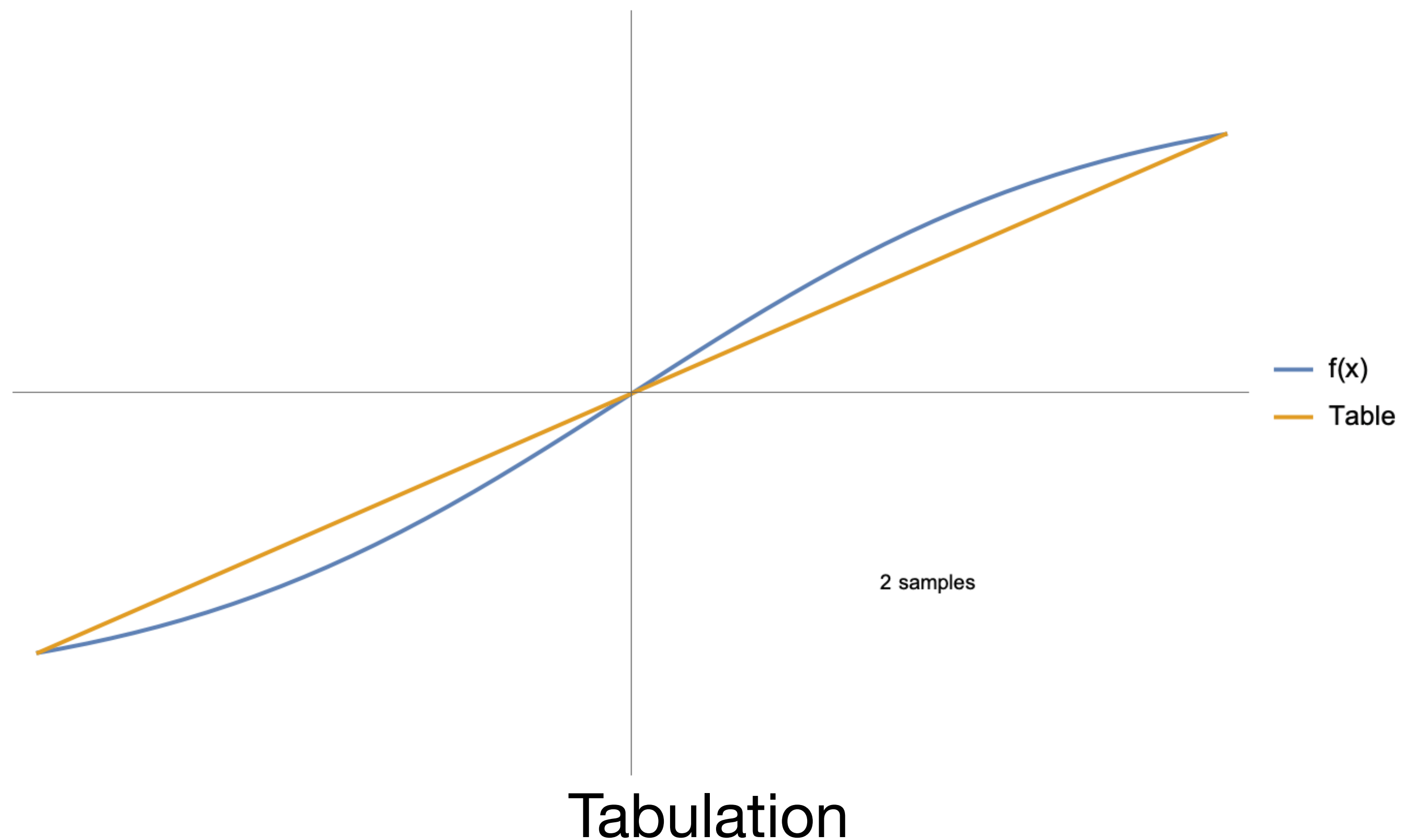
Given an expensive function $f(x)$,
can we find some cheaper way to approximate it?



Rational Approximations

Function Approximation

Given an expensive function $f(x)$,
can we find some cheaper way to approximate it?



Function Approximation

```
./function_approximation
```

```
                    f(x): 53.7539 ms  
    Taylor approximant (2): 2.00192 ms  
    Taylor approximant (3): 3.33926 ms  
    Taylor approximant (3H): 2.66547 ms  
        Pade approximant (2): 10.2758 ms  
        Pade approximant (3): 12.0771 ms  
        Pade approximant (3H): 11.8815 ms
```

Evaluating polynomials in "Horner form" (e.g., 3H variants above) can also be helpful:

$x - 0.5*x*x*x + 0.275*x*x*x*x*x$  $x*(1 + x*x*(-0.5 + x*x*0.275))$

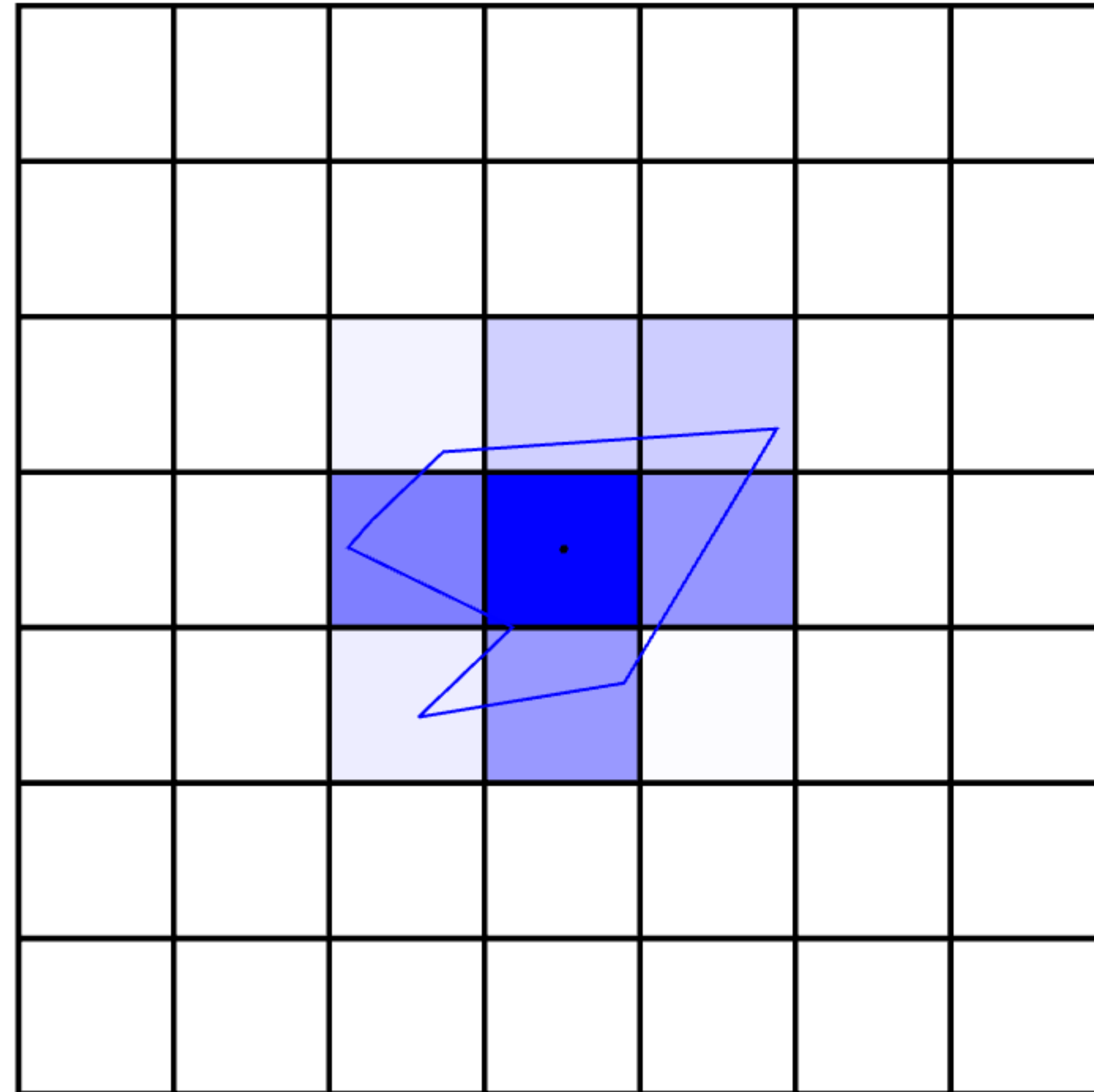
Function Approximation

```
numeratorPolynomialOrder = 5;  
denominatorPolynomialOrder = 3;  
x0 = 0;  
f = PadéApproximant[  
  Sin[Sin[Sin[x]]],  
  {x, x0, {numeratorPolynomialOrder, denominatorPolynomialOrder}}  
]
```

$$\frac{x + \frac{19x^3}{693} + \frac{313x^5}{27720}}{1 + \frac{731x^2}{1386}}$$

Example using Mathematica to compute Padé Approximant

Function Approximation

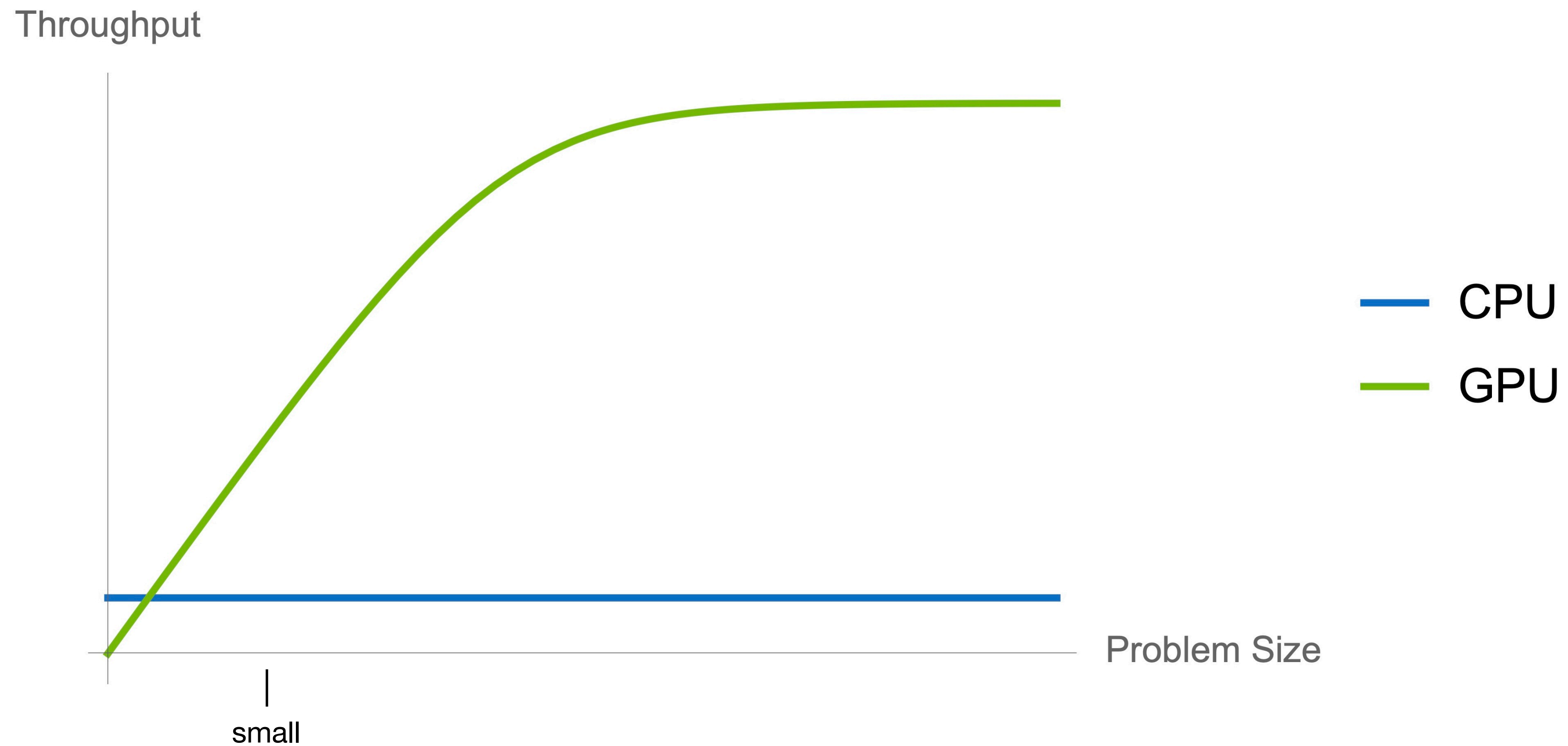


Tabulation can also be useful, but the calculation typically needs to be really expensive to justify the memory accesses!

Function Approximation: Summary

- Taylor / Padé approximants decrease compute but also decrease accuracy
 - Not very helpful for memory-bound kernels!
- Tabulation decreases compute but increases memory traffic
 - Not very helpful for memory-bound kernels!
- Recalculation decreases memory traffic, but increases compute
 - Good for memory-bound kernels
- Curse of Dimensionality

Many Small Kernels



Small kernels don't "fill up" the entire GPU!

Many Small Kernels

Work Items / Problem:

- Large (~100000+): Use a kernel
- Medium (~100-10000): Use an individual block
- Small (~1-100): Use an individual thread

See also: MathDX libraries, CUB

<https://github.com/NVIDIA/CUDALibrarySamples/tree/main/MathDx>

Reduced Precision Arithmetic

Helpful for both compute and memory bound kernels

- Fewer bytes per value decreases total memory accesses
- Operations on reduced precision operands are cheaper

In some cases FP32 can be more than 2x speedup over FP64:

- Compute-bound kernels on consumer hardware (e.g. RTX 5090)
- Kernels with high register pressure

Reduced Precision Arithmetic

Some thoughts:

- In our Conjugate Gradient example, everything was FP64
 - Initial iterates could be FP32, since answer is way off
 - Final iterates could switch to FP64
- FP32-FP32 implementations
 - "Error estimation and correction using the forward CENA method"

Load Imbalance

Consider the following:

```
int tid = threadIdx.x + blockIdx.x * blockDim.x;

int iter = iterations[tid];
double value = in[tid];

for (int i = 0; i < iter; i++) {
    value = sin(value);
}
```


Load Imbalance

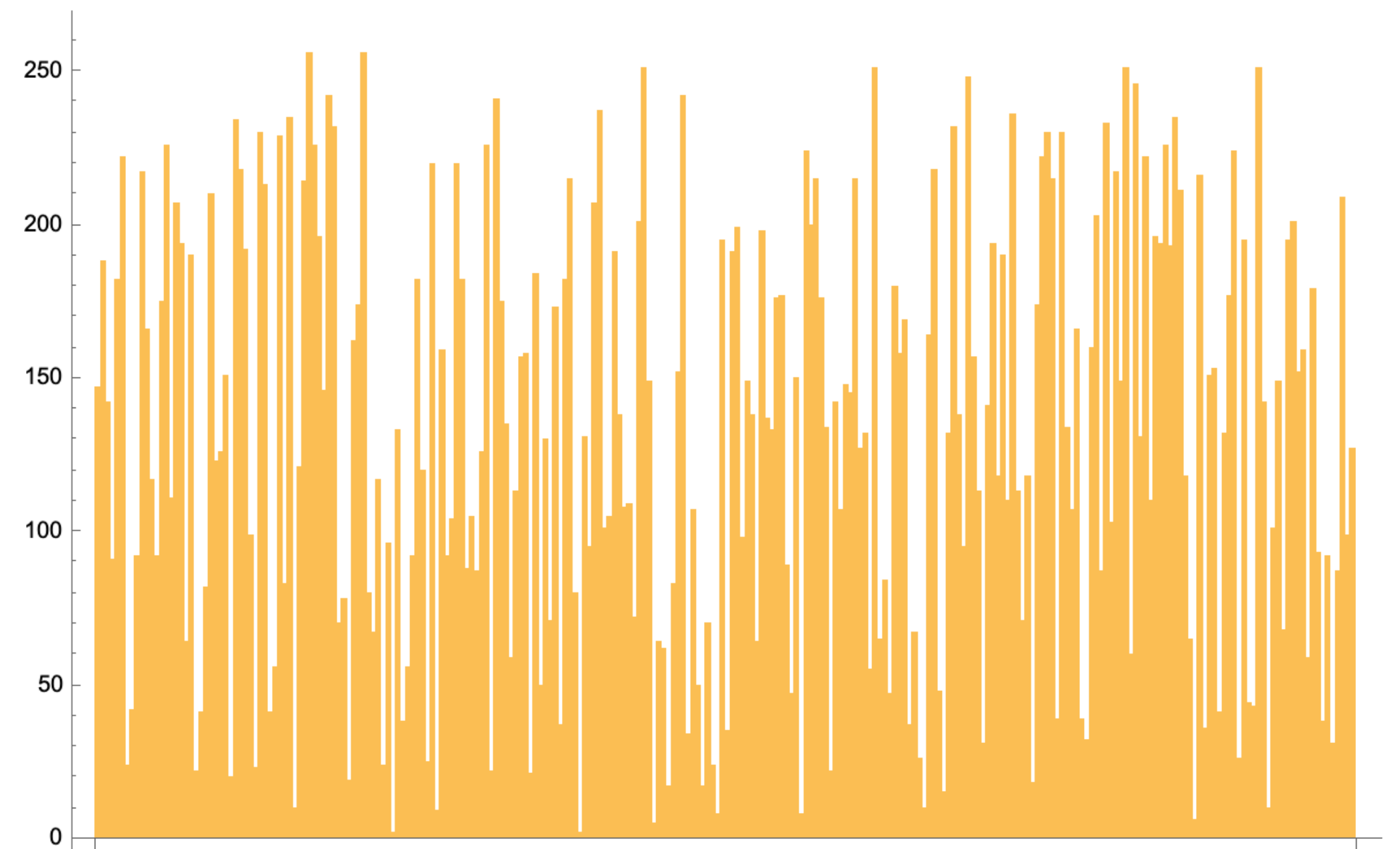
Consider the following:

```
int tid = threadIdx.x + blockIdx.x * blockDim.x;

int iter = iterations[tid];
double value = in[tid];

for (int i = 0; i < iter; i++) {
    value = sin(value);
}
```

iterations array



Load Imbalance

Consider the following:

```
int tid = threadIdx.x + blockIdx.x * blockDim.x;

int iter = iterations[tid];
double value = in[tid];

for (int i = 0; i < iter; i++) {
    value = sin(value);
}
```

iterations array



What if we sorted the data first?

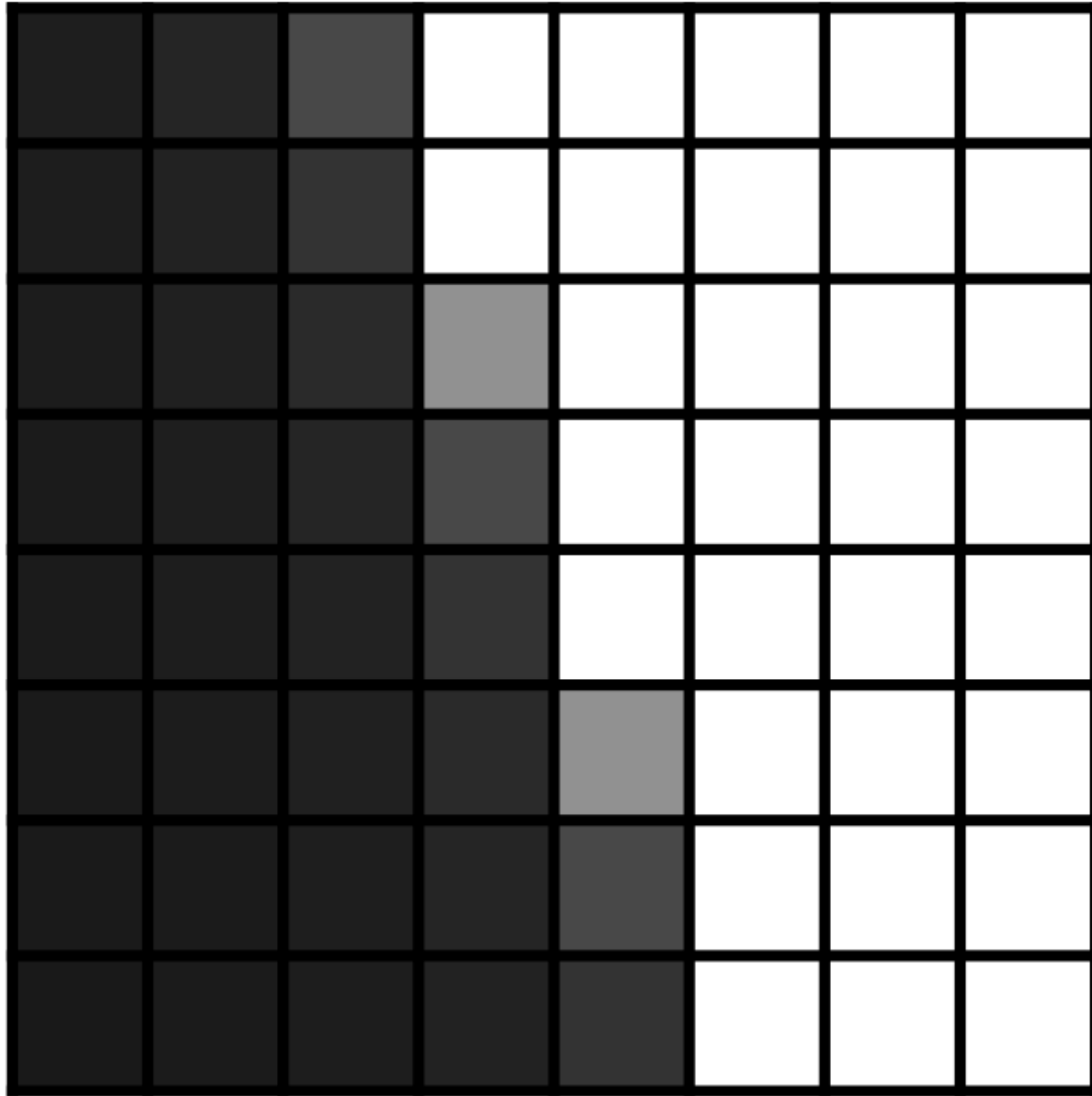
CUB

- distributed as part of CUDA Toolkit
- Open source on GitHub (as part of): <https://github.com/NVIDIA/cccl>
- Optimized GPU-wide algorithms for:
 - Sorting, Reductions, Scan, ...
- Optimized **Block**-wide algorithms for:
 - Sorting, Reductions, Scan, ...

Load Imbalance

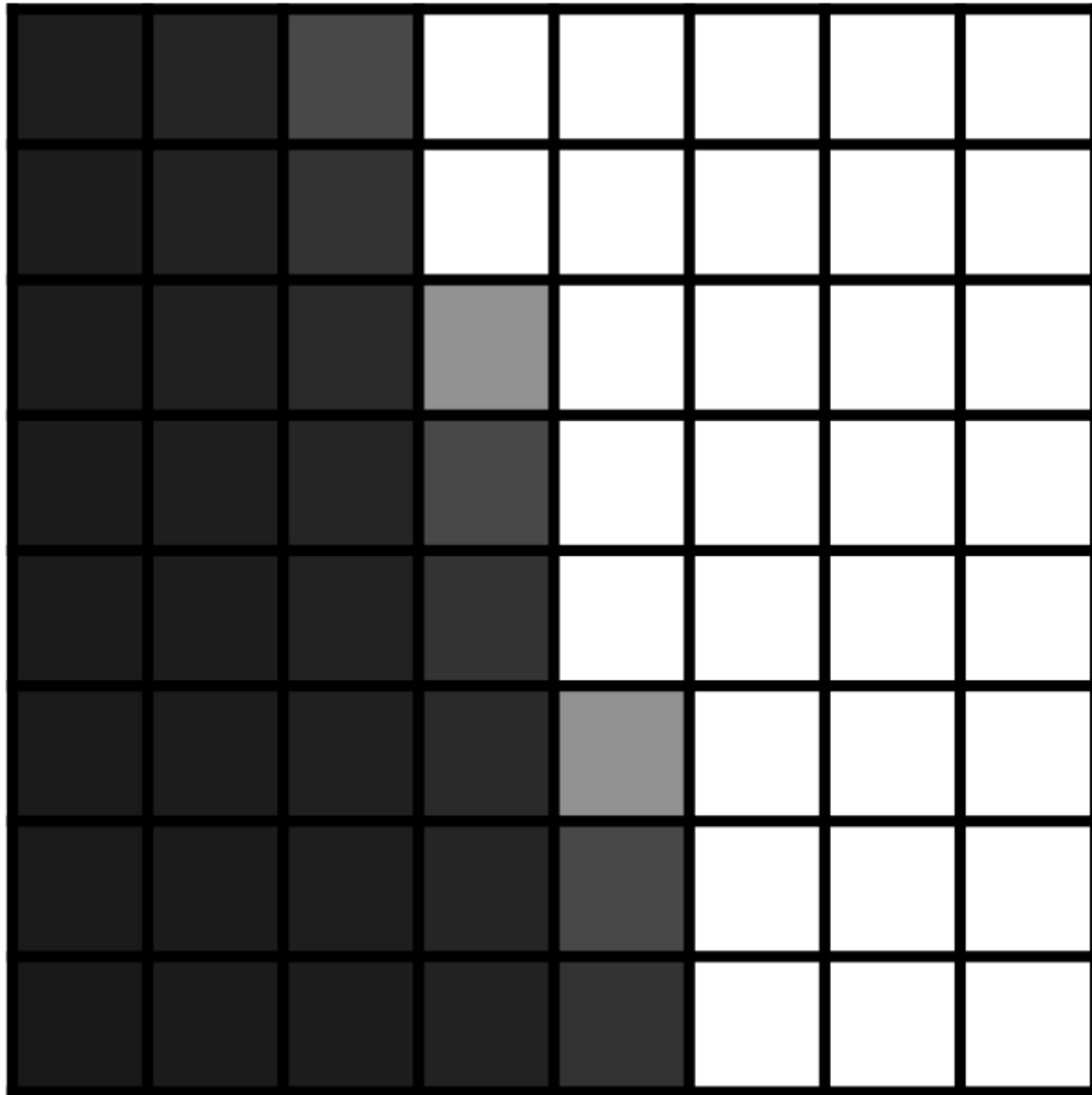
- Exercise: 06_profiling/load_balancing/cub_sort.cu
- Use cub to sort the block's data before processing
 - Relevant [cub documentation](#) on cub::BlockRadixSort
- How does sorting affect the runtime?
- How does the distribution of inputs affect the runtime with and without sorting?

Load Imbalance



```
for (auto [i,j] : grid) {  
    if (is_edge_pixel(i,j)) {  
        do_something_expensive(i,j);  
    }  
}
```

Load Imbalance



```
for (auto [i,j] : grid) {  
    if (is_edge_pixel(i,j)) {  
        edge_pixels.push_back({i,j});  
    }  
}  
  
for (auto [i,j] : edge_pixels) {  
    do_something_expensive(i,j);  
}
```


Thrust

- distributed as part of CUDA Toolkit
- Open source on GitHub (as part of): <https://github.com/NVIDIA/cccl>
- Optimized parallel algorithms (both CPU and GPU) for:
 - Sorting, Reductions, Scan, Set Operations, Search, RNG
- Container classes:
 - e.g., `thrust::host_vector`, `thrust::device_vector`, ...
- Interfaces modeled after C++ standard library
 - e.g., `thrust::transform`, `thrust::reduce`, ...

cuOpt

- Not distributed as part of CUDA Toolkit
- Open source on GitHub: <https://github.com/NVIDIA/cuopt>
- GPU accelerated algorithms for:
 - Linear Programming (LP)
 - Mixed Integer Programming (MIP)
- Recently added support for PDLP
 - <https://developer.nvidia.com/blog/accelerate-large-linear-programming-problems-with-nvidia-cuopt/>

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^\top \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^\top \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^T \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```


cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^T \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^\top \mathbf{x}$ such that $\mathbf{A}\mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^\top \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```


cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^\top \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^T \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

is $x[i]$ continuous or discrete?

cuOpt

A Linear Program in Standard Form:

Maximize $\mathbf{c}^T \mathbf{x}$ such that $\mathbf{A} \mathbf{x} \leq \mathbf{b}$ and $\mathbf{x} \geq 0$

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

no support for binary variables
or 1-hot encoding constraints!

CSR Matrix Format

		Dense						COO						CSR			
		3	0	2	0			0	0	1	1	2	3	3	0	2	4
		0	1	0	2			0	2	1	3	2	0	3	2	1	3
		0	0	5	0			3	2	1	2	5	9	1	5	2	5
		9	0	0	1												
						ind_ptr											
						indices											
						data											

```
cuopt_int_t cuOptCreateProblem(  
    cuopt_int_t num_constraints,  
    cuopt_int_t num_variables,  
    cuopt_int_t objective_sense,  
    cuopt_float_t objective_offset,  
    const cuopt_float_t *objective_coefficients,  
    const cuopt_int_t *constraint_matrix_row_offsets,  
    const cuopt_int_t *constraint_matrix_column_indices,  
    const cuopt_float_t *constraint_matrix_coefficient_values,  
    const char *constraint_sense,  
    const cuopt_float_t *rhs,  
    const cuopt_float_t *lower_bounds,  
    const cuopt_float_t *upper_bounds,  
    const char *variable_types,  
    cuOptOptimizationProblem *problem_ptr  
)
```

image taken from https://hippocampus-garden.com/pandas_sparse/

CSR Matrix Format

Let's work through an "real-world" example:
Generate a conductance matrix for a resistor network given by

```
{  
  {R1, node1_1, node1_2},  
  {R2, node2_1, node2_2},  
  {R3, node3_1, node3_2},  
  ...  
}
```

$$\mathbf{S} = \begin{bmatrix} \frac{1}{R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{1}{R} \end{bmatrix}$$

conductance matrix for an
individual resistor

CSR Matrix Format

If it was a dense matrix we could write ...

```
// create a matrix of zeroes, with as many
// rows/columns as we have nodes in the circuit
DenseMatrix S = zeros(num_nodes, num_nodes);

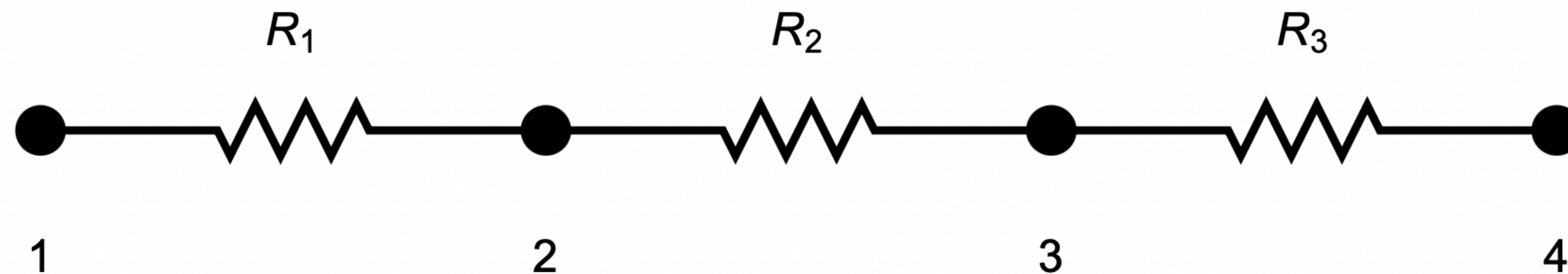
// assemble the conductance matrix, by having
// each resistor
// add its own contribution, as derived above
for (auto [R, i, j] : network) {
    S[i][i] += 1.0 / R;
    S[i][j] -= 1.0 / R;
    S[j][i] -= 1.0 / R;
    S[j][j] += 1.0 / R;
}
```

$$\mathbf{S} = \begin{bmatrix} \frac{1}{R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{1}{R} \end{bmatrix}$$

conductance matrix for an
individual resistor

CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

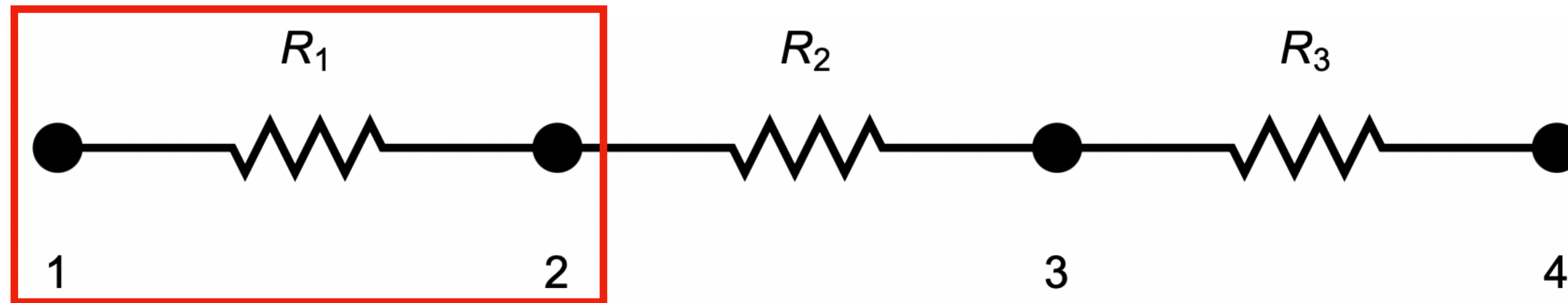


Emit 4 nonzero entries from each resistor, corresponding to its contributions to the conductance matrix (in COO-style format):

```
rows = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
cols = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```


CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

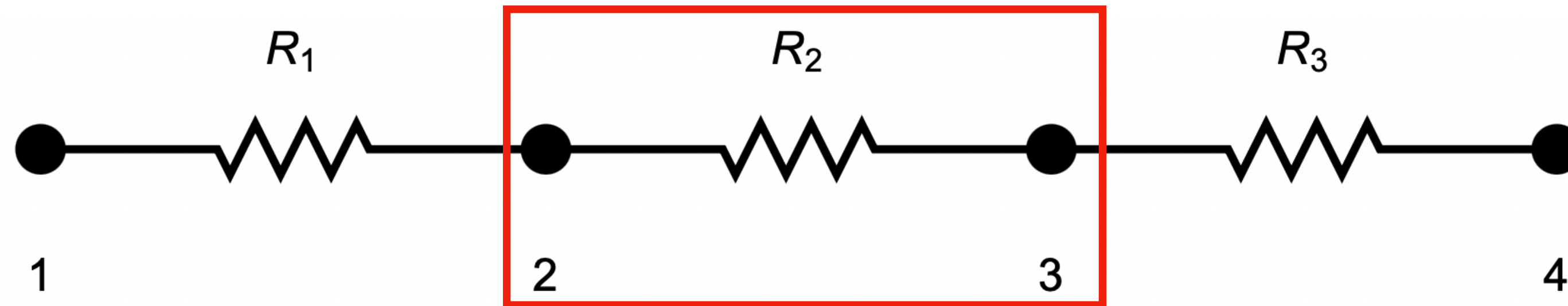


Emit 4 nonzero entries from each resistor, corresponding to its contributions to the conductance matrix (in COO-style format):

```
rows = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
cols = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```

CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

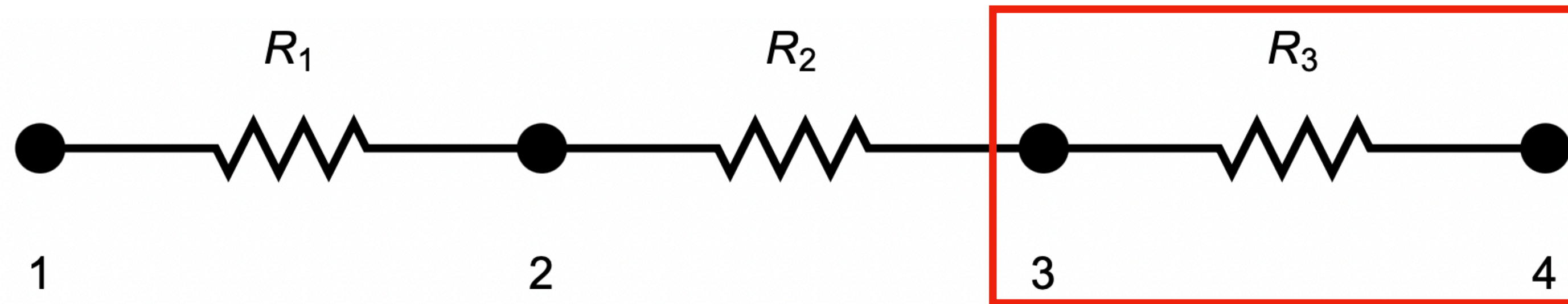


Emit 4 nonzero entries from each resistor, corresponding to its contributions to the conductance matrix (in COO-style format):

```
rows = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
cols = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```

CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

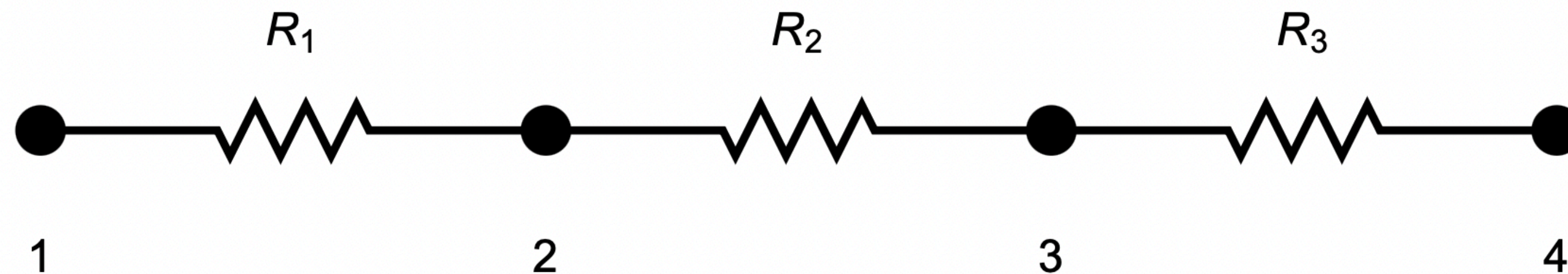


Emit 4 nonzero entries from each resistor, corresponding to its contributions to the conductance matrix (in COO-style format):

```
rows = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
cols = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```


CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

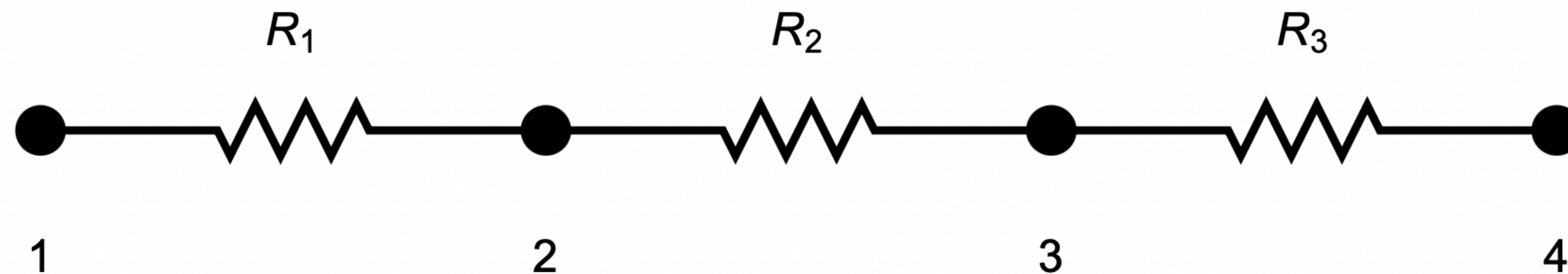


Sort by row/column to group duplicate nonzero positions:

```
sorted_rows = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
sorted_cols = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
sorted_vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```

CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

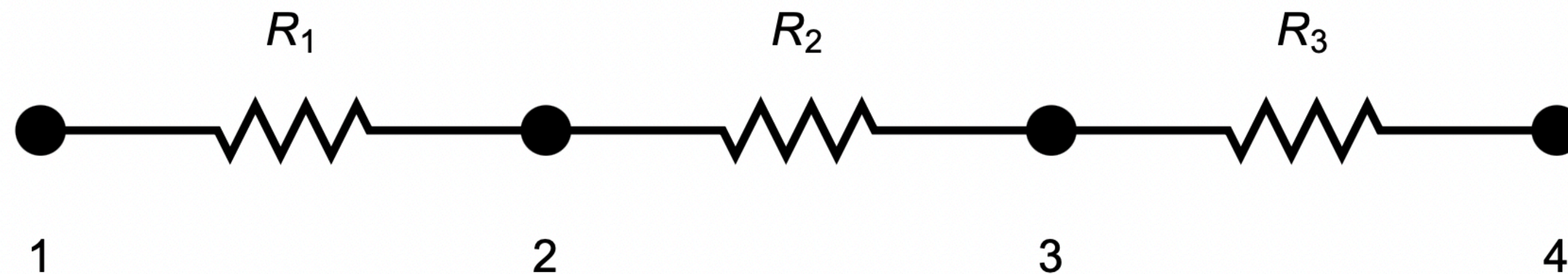


Reduce by key to combine duplicate nonzero entries:

```
sorted_rows = { 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4};  
sorted_cols = { 1, 2, 1, 2, 2, 3, 2, 3, 3, 4, 3, 4};  
sorted_vals = {S1, -S1, -S1, S1, S2, -S2, -S2, S2, S3, -S3, -S3, S3};
```

CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

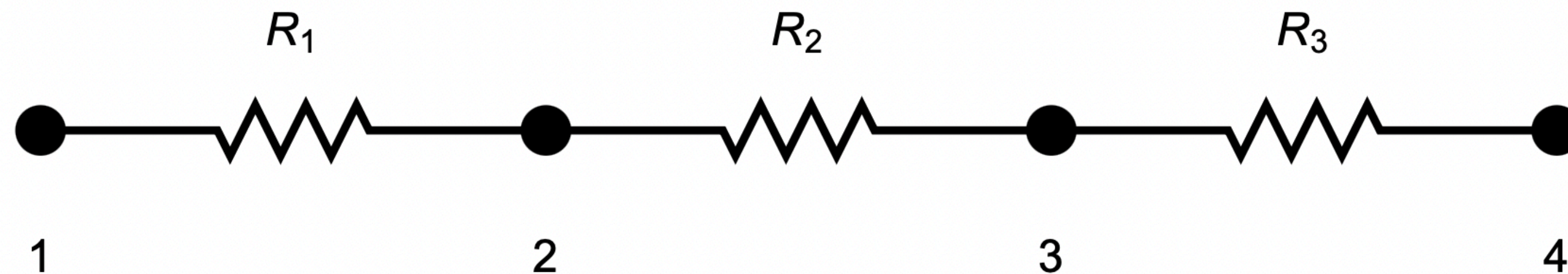


Reduce by key to combine duplicate nonzero entries:

```
reduced_rows = { 1, 1, 2, 2, 2, 3, 3, 3, 4, 4};  
reduced_cols = { 1, 2, 1, 2, 3, 2, 3, 4, 3, 4};  
reduced_vals = {S1, -S1, -S1, S1+S2, -S2, -S2, S2+S3, -S3, -S3, S3};
```


CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:

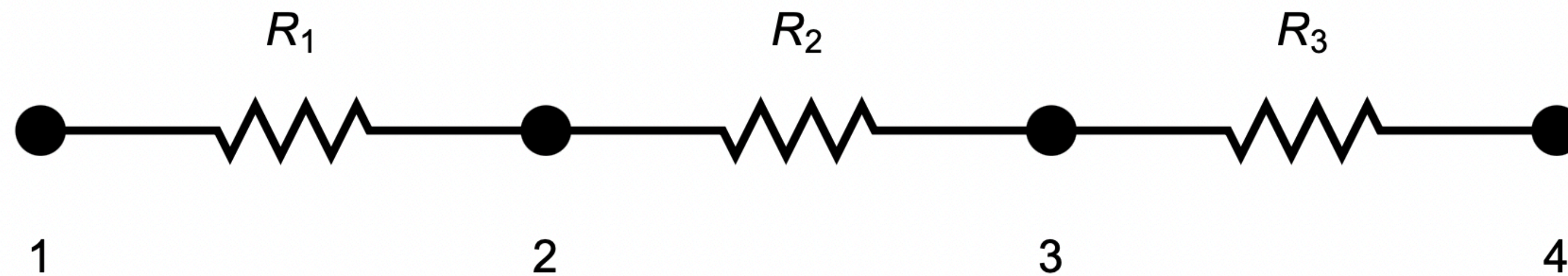


Identify row offsets:

```
reduced_rows = { 1, 1, 2, 2, 2, 3, 3, 3, 4, 4};  
               ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑  
csr_row_ptr  = { 0, 2, 5, 8, 10};  
csr_col_ind  = { 1, 2, 1, 2, 3, 2, 3, 4, 3, 4};  
csr_values   = {S1, -S1, -S1, S1+S2, -S2, -S2, S2+S3, -S3, -S3, S3};
```


CSR Matrix Format

But since we don't want to store zeros, let's try a different approach.
Consider the simple example:



```
csr_row_ptr   = { 0,      2,      5,      8,     10};  
csr_col_ind   = { 1,    2,    1,    2,    3,    2,    3,    4,    3,    4};  
csr_values    = {S1, -S1, -S1, S1+S2, -S2, -S2, S2+S3, -S3, -S3, S3};
```



CSR Matrix Format

To recap, the steps are:

1. Emit COO-triples (row, col, value) for each Resistor
2. Sort by (row, col) pairs to align duplicate nonzero positions
3. Reduce by key to combine duplicate nonzero entries
4. Identify row offsets

CSR Matrix Format

To recap, the steps are:

1. Emit COO-triples (row, col, value) for each Resistor
2. Sort by (row, col) pairs to align duplicate nonzero positions
3. Reduce by key to combine duplicate nonzero entries
4. Identify row offsets

Possible Implementation:

1. Custom Kernel
2. `thrust::sort_by_key`
3. `thrust::reduce_by_key`
4. `thrust::lower_bound`

CSR Matrix Format

Exercise: see 06_profiling/csr_matrix/conductance_matrix.cu

- Implement the custom kernel `emit_coo_values(...)`
 - Emit 4 (row, col, value) entries per resistor
- Profile with Nsight systems
 - `nsys profile -o conductance_matrix ./conductance_matrix`
 - Where is time being spent in these different steps?
 - What are some possible optimizations?

CSR Matrix Format

Let's inspect `conductance_matrix.nsys-rep`

CSR Matrix Format

Let's look a specific LP relevant to your application: Minimum-Cost Flow

The definition of the problem is to minimize the **total cost** of the flow over all edges:

$$\sum_{(u,v) \in E} a(u,v) \cdot f(u,v)$$

with the constraints

Capacity constraints: $f(u,v) \leq c(u,v)$

Skew symmetry: $f(u,v) = -f(v,u)$

Flow conservation: $\sum_{w \in V} f(u,w) = 0$ for all $u \neq s, t$

Required flow: $\sum_{w \in V} f(s,w) = d$ and $\sum_{w \in V} f(w,t) = d$

CSR Matrix Format

Let's inspect a similar kernel for generating the COO entries for this kind of network:

`06_profiling/csr_matrix/network_flow.cu`

CSR Matrix Format

Let's inspect a similar kernel for generating the COO entries for this kind of network:

`06_profiling/csr_matrix/network_flow.cu`

How could we change this kernel to improve its runtime?

CSR Matrix Format

Only some of the fields in the Arc class are actually **necessary** for the COO entries.

The cost and capacity fields are relevant for the objective and constraint enforcement, but that happens elsewhere.

Idea: try separating some of the data types to avoid loading unnecessary data

CSR Matrix Format

```
sam@gouda:~/code/Intro-to-CUDA/build$ ./network_flow
```

```
emit_coo_values time: 0.25664 ms
```

```
sam@gouda:~/code/Intro-to-CUDA/build$ ./network_flow_solution
```

```
emit_coo_values time: 0.182464 ms
```


CSR Matrix Format

Summary:

- Data Layout matters!
- "Array of Structs" vs "Struct of Arrays"
 - think about data access patterns
- In fact, CSR format is somewhat wasteful here
 - Only really need 1 bit to store coefficients for this problem statement, but we store 8 bytes (double)

back to cuOpt

Performance Comparison on "netflow.problem"

CPU Algorithm 1: 18.7s

CPU Algorithm 2: 7.5s

cuOpt (H100, PDL P): 1.7s

Compiler Intrinsic

see: [docs](#)

Some of my favorites:

popcnt, clz, ctz, brev

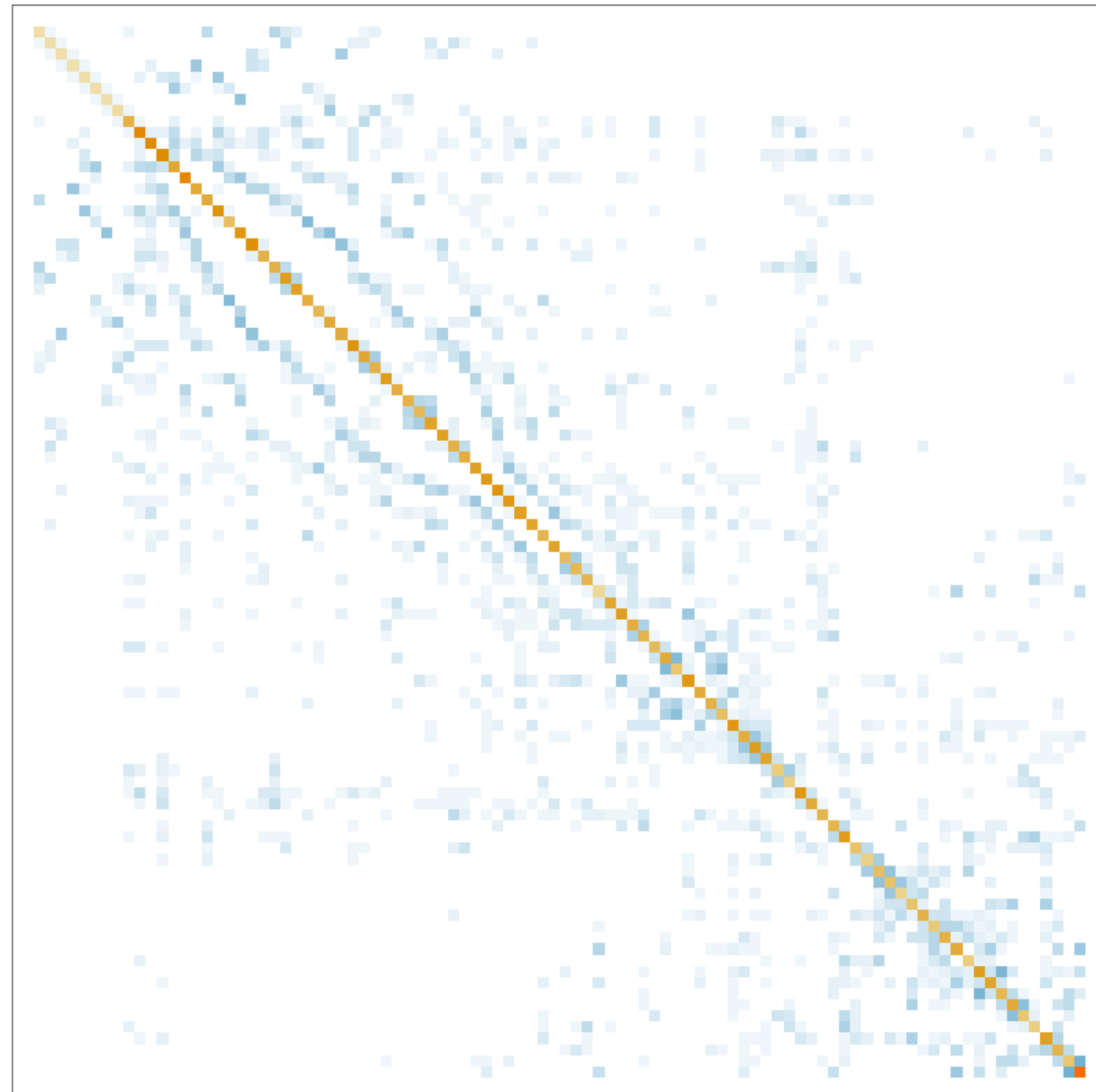
(useful for warp ballots)

cuDSS

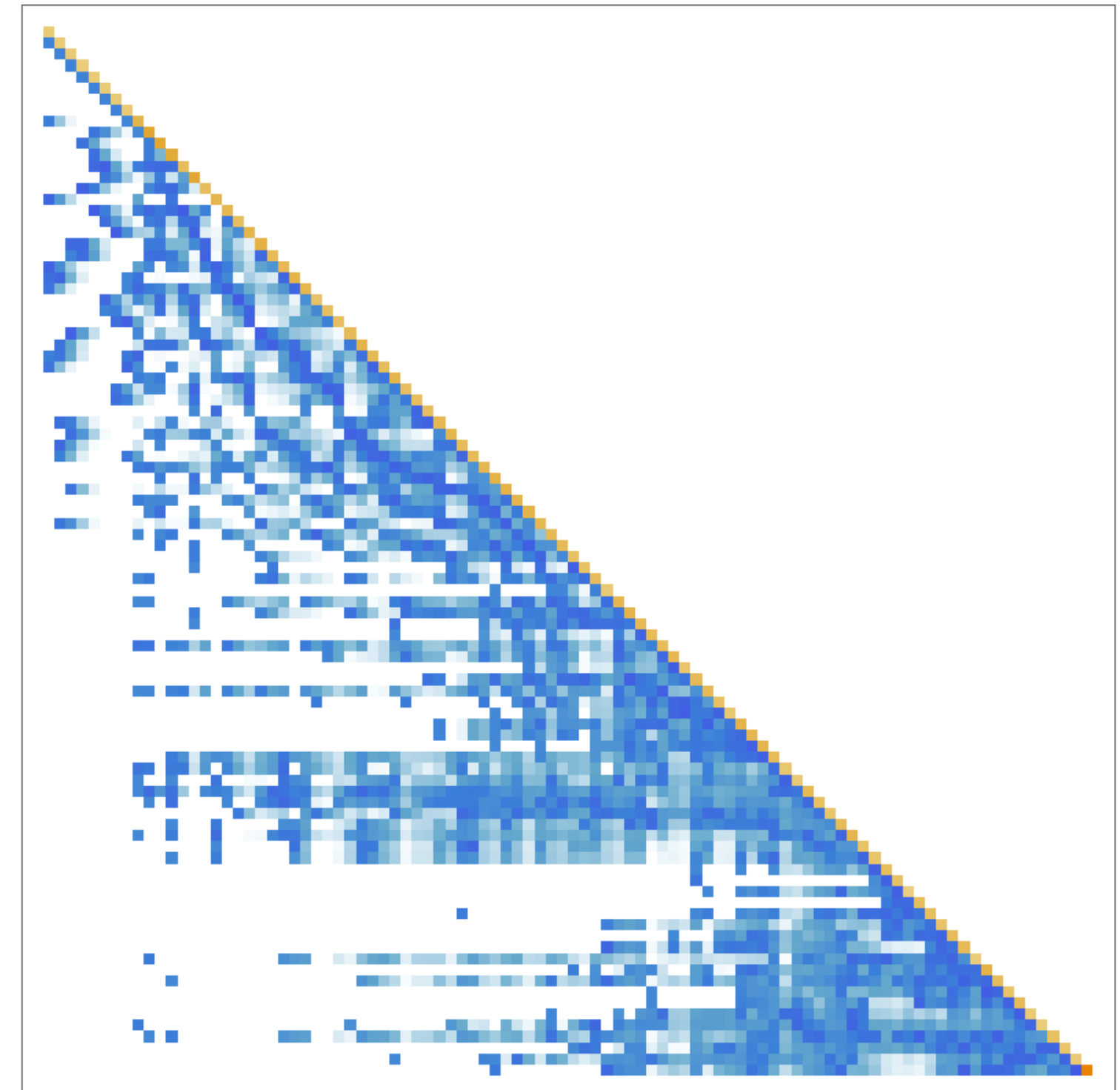
- Not distributed as part of CUDA Toolkit
- Binaries are available through NVIDIA: <https://developer.nvidia.com/cudss>
- GPU-accelerated direct methods for solving systems of equations
 - Matrix Factorizations (Cholesky, LU, etc)
 - Matrix Reordering methods
- Effectively replacing cuSolver

cuDSS

Factorized Matrix may not be sparse!



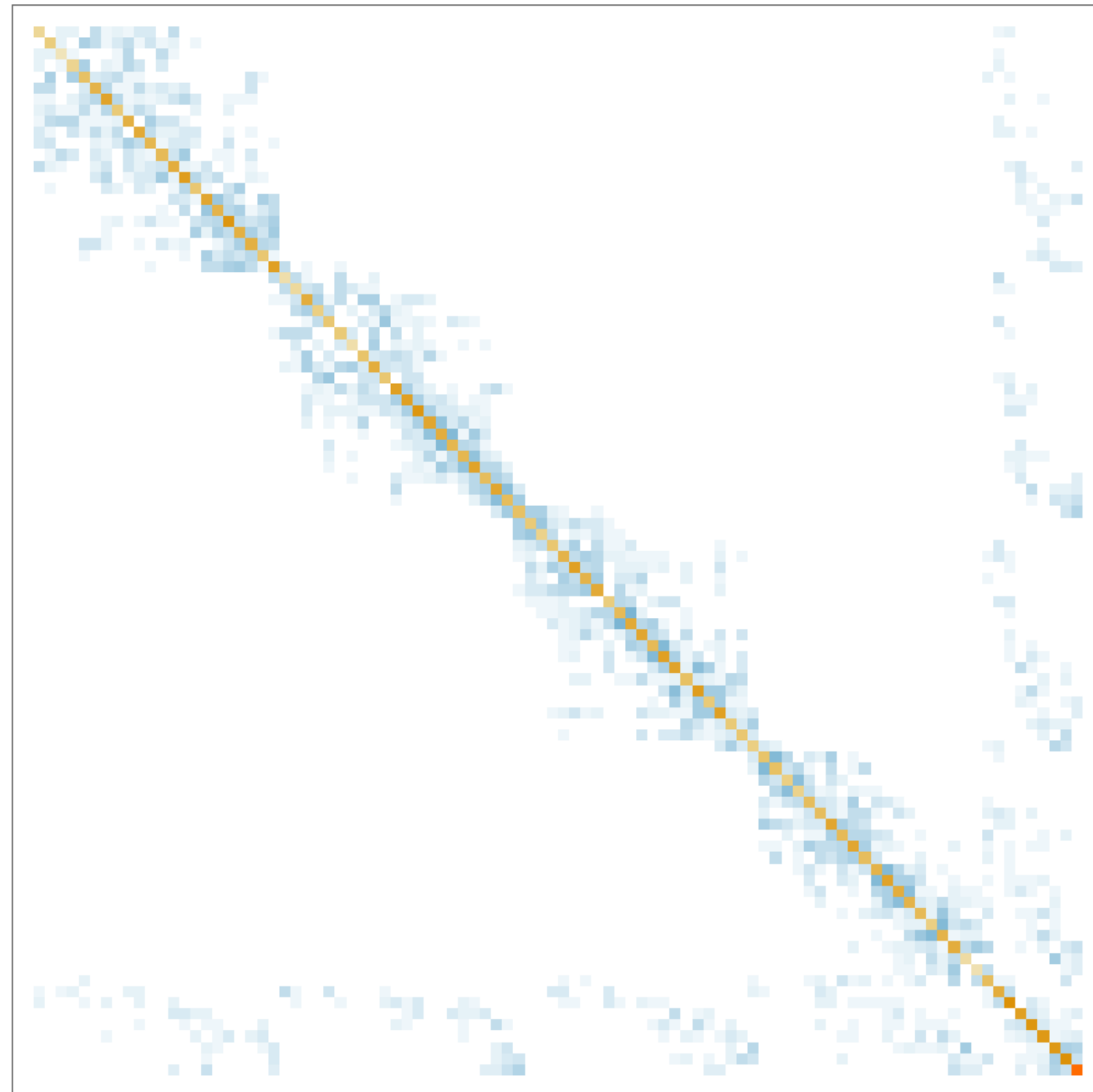
Original Matrix



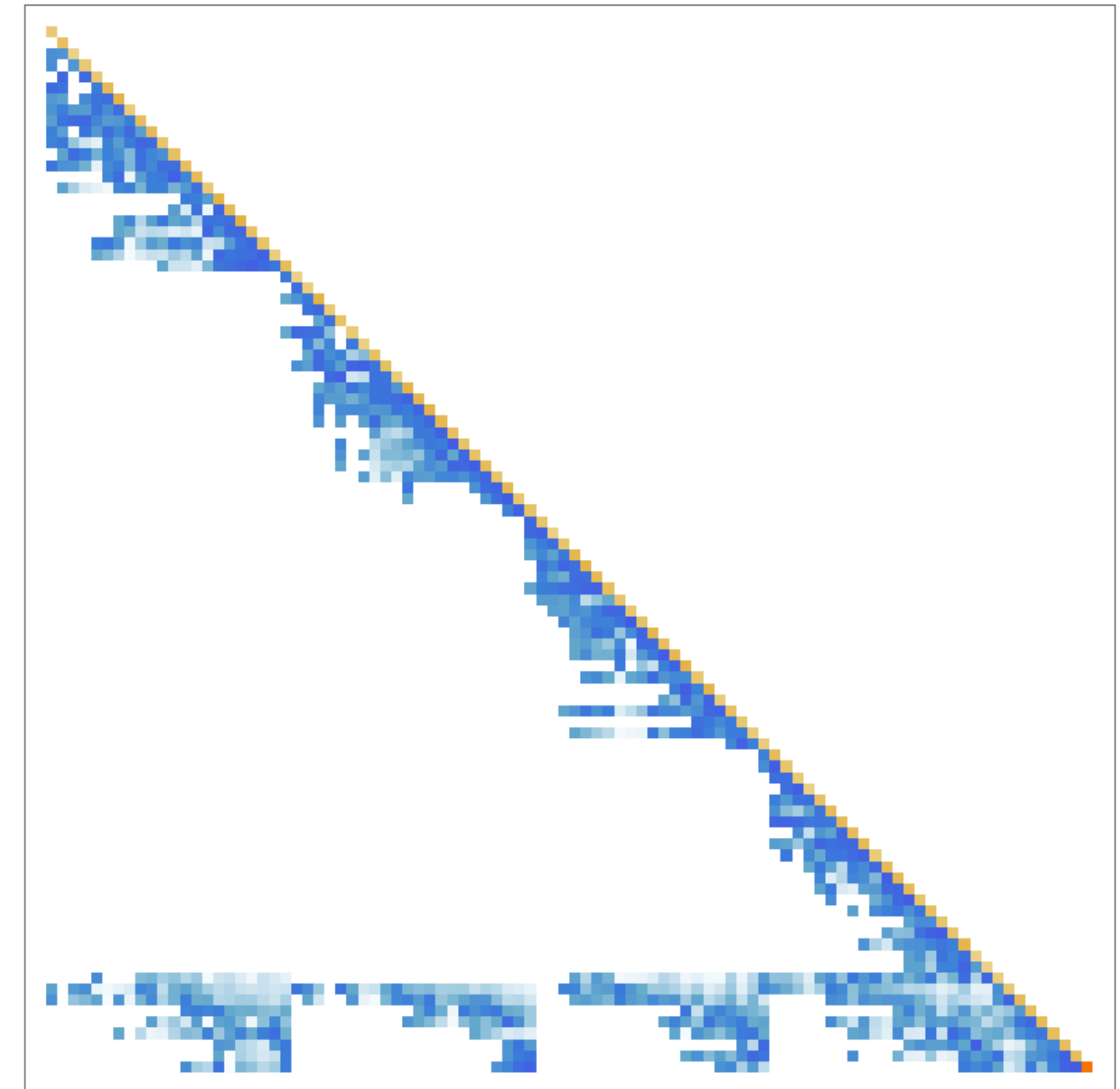
Cholesky Factor

cuDSS

Permutation of the rows / columns reduces fill-in and increases parallelism



Permuted Matrix



Cholesky Factor

cuDSS

Steps:

1. Create Sparse Matrix (CSR format)
2. Perform "Symbolic Factorization"
 1. Identifies good permutation, allocates memory
3. Perform "Numeric Factorization"
4. Use Factorized matrices to solve equations

cuDSS

Steps:

1. Create Sparse Matrix (CSR format)

**Expensive, but
potentially reused**

2. Perform "Symbolic Factorization"

1. Identifies good permutation, allocates memory

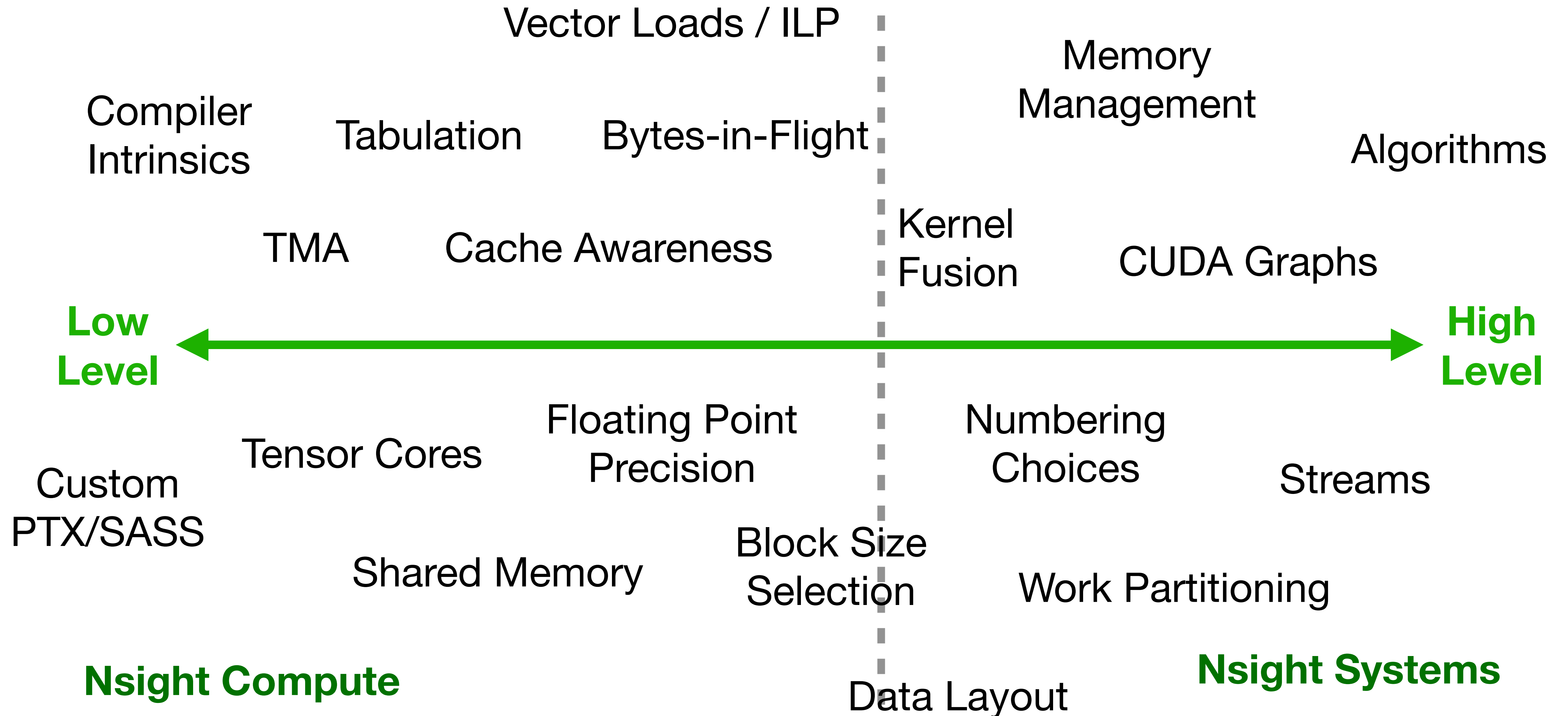
3. Perform "Numeric Factorization"

4. Use Factorized matrices to solve equations

AMGX

- Not distributed as part of CUDA Toolkit
- Open source on GitHub: <https://github.com/NVIDIA/AMGX>
- GPU-accelerated iterative methods for solving systems of equations
 - Algebraic Multigrid preconditioners
 - Krylov methods (PCG, GMRES, BICGSTAB)
 - Smoothers (Gauss-Seidel, Jacobi, ...)

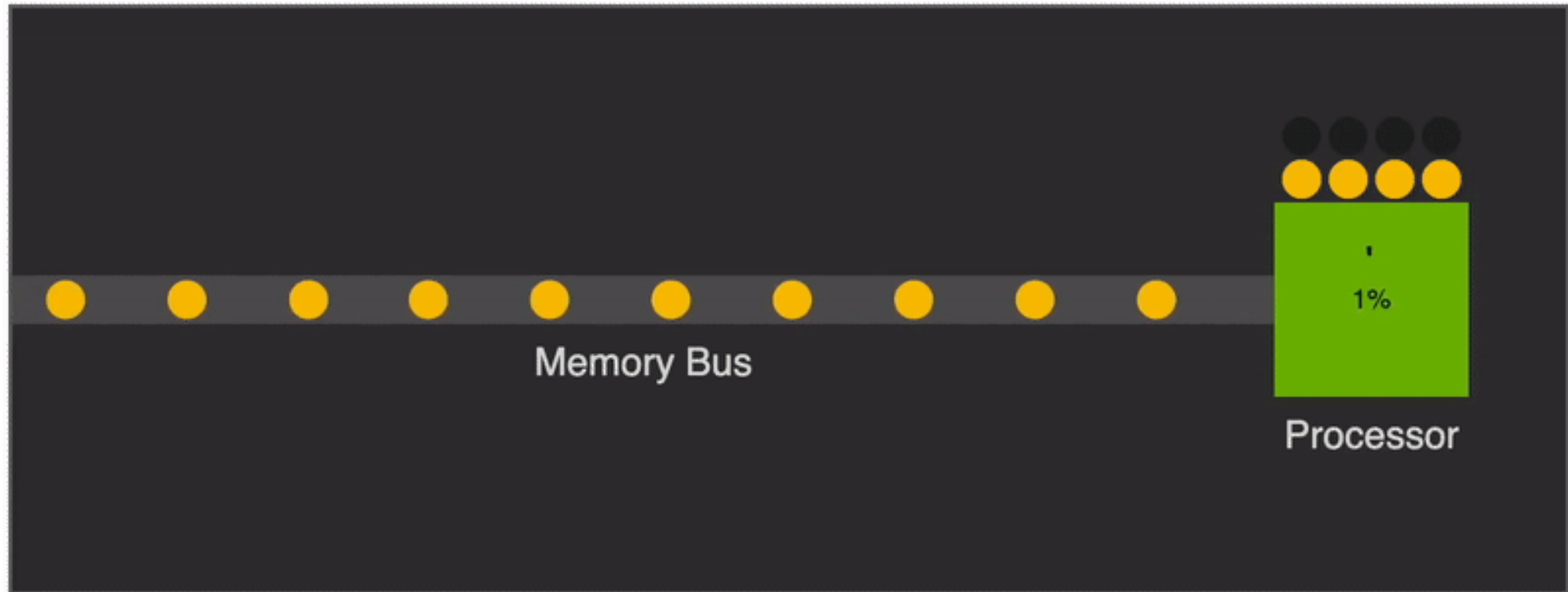
Application Optimizations



Summary: Kinds of Bottlenecks

- Compute Bound: **High** Compute Throughput, **Low** Memory Throughput
 - Time to do computation outweighs time to move data
- Memory Bound: **Low** Compute Throughput, **High** Memory Throughput
 - Time to move data outweighs time to do computation
- Latency Bound: **Low** Compute Throughput, **Low** Memory Throughput
 - Not enough concurrency to hide latency (typically)

Summary: Memory-Bound Kernels



Summary: Memory-Bound Kernels

- Which memory subsystem is the bottleneck?
 - L1, L2, DRAM
- What are my data access patterns like?
 - coalesced / random, structured / unstructured
- How is my data laid out?
 - flat, tree-like, AoS, SoA, AoSoA
- Do I have enough bytes-in-flight?
 - occupancy, vector loads, ILP, identify stalls
- Are multiple memory-bound kernels executed in sequence?
 - consider kernel fusion, or sizing problems to fit in L2\$
- Are all the data accesses necessary?
 - some quantities could be recalculated, rather than loaded

Summary: Compute-Bound Kernels



Summary: Compute-Bound Kernels

- Which kind of instructions are the bottleneck?
 - integer, floating point, LSU
- How accurate do my answers need to be?
 - FP32/FP64, numerical approximations
- Are all the calculations necessary?
 - save/load expensive quantities
 - table-lookup
 - transcendental functions
 - integer division and modulo
- Am I multiplying lots of dense matrices?
 - tensor cores
- See also: compiler intrinsics / inline PTX

Summary: Compute-Bound Kernels

- Which kind of instructions are the bottleneck?
 - integer, floating point, LSU
- How accurate do my answers need to be?
 - FP32/FP64, numerical approximations
- Are all the calculations necessary?
 - save/load expensive quantities
 - table-lookup
 - transcendental functions
 - integer division and modulo
- Am I multiplying lots of dense matrices?
 - tensor cores
- See also: compiler intrinsics / inline PTX

Training Summary

- Talked about GPU Hardware
 - Building mental model for different systems
 - Thread Hierarchy, Cache, Registers
- Talked about Kernels
 - concurrency, race conditions, shared memory
 - function annotations, CUDA libraries
- Performance Modeling and Optimizations
 - Profile, Identify Bottlenecks, and Iterate
 - Focus on the important topics to get most return on time investment

Thanks!